MADRE Agentic System

· Model-Agnostic · Delayed · Reasoning · Effort · [Beta v0.1]

Project Definition and Architecture Documentation

Scvria Sofware

May 25, 2026

Contents

Front Matter

Abstract

1.1 Executive Definition

> **MADRE** *is a local-first, model-agnostic runtime kernel for building agentic systems that preserve user sovereignty while coordinating reasoning modules, module-owned agents, governed context, bounded actions, known material, evaluated learning, recovery, and inspectable execution.*

MADRE Agentic System[Beta v0.1] exists because the current AI market is normalizing a dangerous bargain: users receive convenient language interfaces, while their data, dependency chain, costs, and decision power move into remote platforms they do not control. That bargain is technically unnecessary and politically unacceptable for many individuals, teams, public-interest organizations, researchers, creators, and small developers.

The project does not treat a language model as a complete intelligence. A model is a powerful linguistic function: useful, fallible, statistically trained, expensive in context, and unsafe when it is allowed to become the whole system. **MADRE** puts that function inside an engineered runtime. The runtime decides what may be known, what may be sent, what may be stored, what may be acted on, and what must wait for stronger evidence or human authority.

The product objective is one coherent assistant experience backed by a durable kernel. Visible interaction can remain fast and conversational while module-local reasoning and reflection lanes decompose, schedule, check, record, and resume deeper work. This is the practical meaning of Model Agnostic Delayed Reasoning Effort: useful local intelligence should come from architecture, context discipline, and bounded autonomy, not from blind dependence on one remote provider or one large model.

MADRE is therefore not a chatbot, not a prompt collection, not a wrapper around one vendor API, and not a task-management application. It is the runtime layer that applications use to coordinate models, internal actions, conversation and knowledge records, policies, and interfaces without surrendering the user's private domain to cloud-first AI infrastructure.

Specification Scope

MADRE is a local-first, model-agnostic runtime kernel for constructing agentic systems whose useful behavior is produced by software architecture rather than by treating language models as autonomous intelligence. The runtime assigns a Default interaction module, permits its `ModuleMainAgent` to provide

foreground continuity, governs context before `ModelAction` or other actions, keeps `KnowledgeRecord` handling inspectable, records local traces through `RuntimeJournal`, and requires learning to pass through evaluation, promotion, and rollback boundaries.

This specification defines the research, product, and architecture baseline for MADRE. It states the system problem, expected contribution, stakeholder concerns, requirement domains, architecture views, safety model, policy model, and evaluation surface used to assess runtime behavior. The product is an open, extensible foundation for private and affordable agentic software on ordinary local devices, with optional remote services and integrations treated as governed risk boundaries rather than defaults.

The scope of **MADRE** comprises runtime purpose, system boundary, requirement families, architecture obligations, risk model, quality attributes, validation strategy, and product-level authority semantics.

The design identifiers D-001 through D-013 are internal traceability anchors inside this dossier. They identify product rationale, runtime obligations, safety boundaries, and evaluation surfaces at architecture level without making prior side specifications authoritative.

Canonical Terms

Term	Meaning in MADRE
`MadreKernel`	Deterministic local coordinator that registers `ReasoningModule` objects, assigns the Default interaction module, routes requests, schedules work, coordinates resources, applies policy, and records runtime activity through `RuntimeJournal`.
`ReasoningModule`	Governed reasoning unit for a bounded domain or scope. It owns module records, workflows, routines, actions, agents, knowledge and learning boundaries, and allowed model/inference scope.
Default interaction module	Kernel registration role held by a compatible `ReasoningModule` that receives ordinary interaction first; it is not a separate module type.
`MADREAgent` / `ModuleMainAgent`	A `MADREAgent` is a concrete module-owned runtime actor constrained by `ModelBinding`; `ModuleMainAgent` is the role held by one such agent, possibly using a `ComplexAgent` profile.

Term	Meaning in MADRE
`SimpleAgent`, `TwinAgent`, `ComplexAgent`	Agent profiles for direct execution, review/evaluation over a produced result, and foreground/reasoning/reflection capability respectively; none gains authority outside its owning module.
`ReasoningPlan`, `Workflow`, `AgentRoutine`	A request-derived reasoning plan may select reusable module workflow and agent routine capabilities without turning either into a second plan or a new authority layer.
Access surface	A CLI, local console, GUI, speech surface, or application interface that exposes the kernel without becoming runtime authority.
`WorkRecord`, `RuntimeEvent`, `RuntimeJournal`	`WorkRecord` tracks delayed-work lifecycle; `RuntimeEvent` is append-only runtime history; `RuntimeJournal` is the local trace mechanism that stores or references them and related payloads.
`PolicyDecision`	Explicit allow, block, or authorization-required outcome for an affected action or data class and authority boundary.
Foreground lane	A possible `ComplexAgent` capability used by a module's `ModuleMainAgent` to preserve flow, clarify intent, answer within safe context, and create delayed work when stronger evidence is required.
Reasoning lane	Durable module-local work used for analysis, evidence gathering, verification, internal action invocation, or higher-effort inference.
`ContextSource` / `ContextBundle`	Source descriptor and bounded, classified, minimized, revocable selection of material prepared for an agent request, action, or inference execution.

Term	Meaning in MADRE
`AgentAction`, `ModelAction`, `ModelBinding`, `InferenceProfile`, `InferenceBackend`	An executable capability and its inference-backed specialization; a model-backed action executes only through the binding allowed for its module-owned agent, with configured and replaceable inference execution.
`InferenceRecord` / `InferenceOutput`	One recorded inference execution and its generated material. `InferenceOutput` is not truth, policy, memory, knowledge, behavior, routing, authority, or a candidate by itself.
`KnowledgeRecord` / `KnowledgeCandidate`	Known material held by a module with provenance, scope, sensitivity, intended use, truth-level/authority, correction, and lifecycle metadata; only a pending knowledge-boundary change is a `KnowledgeCandidate`.
`LearningCandidate` / `ConversationRecord`	An evaluated proposed module improvement and retained interaction material respectively. Saving known material is not learning, and conversation is not knowledge by default.
Artifact	A materialized output such as a file, document, code patch, report, image, dataset, PDF, snapshot, or final buffer/document. Ordinary conversation text is not an artifact unless materialized.

Specification Baseline

Version	**Status**
Beta v0.1	Research, product, and architecture specification baseline for local-first, model-agnostic delayed reasoning. The baseline covers product requirements, architecture obligations, safety boundaries, traceability, and evaluation criteria.

Research Foundation

2.1 Research Problem

Modern language-model systems often combine private data, model calls, action execution, retained interaction, known material, and remote infrastructure inside opaque conversational interfaces. This makes useful demonstrations easy to produce, but it weakens local ownership, traceability, autonomy control, error recovery, and the ability to replace models or providers.

MADRE addresses this as a Software Engineering problem: how to design a local-first runtime in which language models remain fallible replaceable functions, while the software system owns policy, context, knowledge, internal actions, runtime trace, recovery, and learning boundaries. The research problem is not to prove that models reason, but to define an architecture that extracts useful language behavior without surrendering system authority to `InferenceOutput`.

2.2 Research Questions

ID	Research question
RQ-1	How can a local-first runtime separate immediate interaction from delayed reasoning without losing continuity, accountability, or user control?
RQ-2	How can model-agnostic contracts prevent provider-specific assumptions from entering policy, knowledge handling, action authority, and truth management?
RQ-3	What traceability structure is sufficient to connect **MADRE** 's product claims, stakeholder concerns, runtime requirements, architecture decisions, and validation evidence?
RQ-4	How can context, `ConversationRecord`, `KnowledgeRecord`, `KnowledgeCandidate`, and `LearningCandidate` remain useful while preserving provenance, truth authority, minimization, revocation, correction, promotion, and rollback boundaries?
RQ-5	What integrated behavior demonstrates that **MADRE** is a governed runtime rather than a conversational wrapper?

2.3 Hypotheses

ID	Hypothesis
H-1	Module-local foreground, reasoning, and reflection lanes can preserve conversational responsiveness while improving reliability for tasks that require evidence, verification, action execution, or stronger context.
H-2	Model-agnostic contracts can reduce provider lock-in when policy, knowledge handling, context, and action execution remain owned by the runtime.
H-3	Context governance with provenance, classification, minimization, and revocation can reduce leakage and misuse without eliminating useful specialized reasoning.
H-4	Evaluated `LearningCandidate` promotion can improve system adaptation while preventing `InferenceOutput` or ordinary `KnowledgeRecord` storage from silently rewriting protected behavior or factual authority.
H-5	`RuntimeJournal`, `WorkRecord`, and append-only `RuntimeEvent` history can make delayed reasoning inspectable and recoverable enough for ordinary local operation.

2.4 Expected Contributions

MADRE 's expected contribution is a reference architecture for local-first, model-agnostic delayed reasoning. The contribution combines module-owned agents, optional Complex Agent lane profiles, governed context bundles, explicit `ModelAction` and internal action contracts, `InferenceRecord`/`InferenceOutput` separation, controlled learning, and traceable recovery paths.

The novelty is not a new language model. It is the engineering arrangement that keeps language models useful while preventing them from becoming sources of truth, authority, policy, knowledge, or action. The architecture is intended to be useful for personal assistants, developer systems, organizational knowledge workers, domain systems, and local interfaces that require control over private knowledge.

2.5 Novelty And Validation Strategy

The first research claim is that a useful agentic system can be validated as a software architecture before it is validated as a broad automation product. **MADRE** 's novelty is the conjunction of local-first execution, replaceable inference and action boundaries, delayed reasoning, governed context, explicit `PolicyDecision` outcomes, evaluated learning promotion, and journaled recovery.

The validation strategy examines whether a local request can enter through bounded access surfaces, be triaged without unsupported claims, become a durable `WorkRecord` when stronger evidence is needed, obtain a governed context bundle, cross a replaceable `InferenceBackend`, invoke a bounded internal action after a permitting `PolicyDecision`, produce an `InferenceOutput` that remains non-authoritative, expose `RuntimeJournal` trace evidence, and support correction, invalidation, supersession, rollback, or deletion/detachment.

The validation strategy treats negative paths as first-class evidence. Prompt or source injection must remain source data. Known material used with an incorrect `truthLevel` or `truthAuthority` must be rejected or corrected. Sensitive or unrelated material must be excluded from context. `InferenceOutput`

must not become truth, policy, action authority, memory, knowledge, behavior, routing, or a candidate by itself. A `KnowledgeCandidate` changes a knowledge boundary only through promotion, and a `LearningCandidate` changes module behavior only after defined evaluation and promotion.

2.6 Related Work And Evidence Map

Evidence family	Relevance to MADRE	Specification use
Local-first software	Establishes user ownership, offline usefulness, privacy, longevity, and optional synchronization as product concerns.	Supports data sovereignty and local-first architecture claims.
Requirements engineering	Provides structure for measurable requirements, acceptance criteria, verification methods, and source traceability.	Supports the SRS and traceability model.
Architecture description	Provides stakeholder, concern, viewpoint, view, correspondence, and decision-rationale concepts.	Supports multi-view architecture description.
LLM and agent security	Identifies prompt injection, sensitive disclosure, supply-chain risk, data poisoning, excessive agency, vector weakness, misinformation, and resource consumption.	Supports threat and policy modeling.
Software quality models	Provide vocabulary for reliability, maintainability, portability, security, performance efficiency, usability, and quality-in-use evaluation.	Supports the quality attribute model.
AI evaluation research	Provides multi-metric and evidence-oriented evaluation patterns for generated material, retrieval quality, and agentic task performance.	Supports MADRE-native benchmark families.

Product Definition

3.1 Executive Definition

MADRE is a runtime foundation rather than a single assistant interface. The kernel assigns ordinary interaction to a Default interaction module, records delayed lifecycle and runtime history locally, and keeps policy, knowledge, action, and recovery authority outside inference. Language models run through module-constrained `ModelAction` execution; each `InferenceOutput` is generated material from an `InferenceRecord`, not authority or a candidate by itself.

3.2 Product Rationale

MADRE Agentic System

Rationale	Product implication
Local control	Private data, local knowledge, and sensitive work remain under local authority unless a policy-governed transfer is explicitly permitted.
Model substitutability	No model provider, host, prompt convention, or output format defines runtime truth, policy, knowledge, or action authority.
Delayed reasoning	Work that requires evidence, verification, resource control, or stronger context is represented as a traceable `WorkRecord` rather than as an unsupported foreground answer.
Context governance	Context is selected, minimized, classified, sourced, scoped, provenanced, and revocable before it is provided to a `ModelAction`, internal action, or reasoning task.
Bounded action use	Internal actions, external actions, knowledge change, and workflow execution are allowed only through declared authority boundaries, `PolicyDecision` outcomes, trace evidence, and recovery expectations.
Known material and learning	A `KnowledgeRecord` is not factual truth by default; a `KnowledgeCandidate` is reserved for pending knowledge-boundary change, while a `LearningCandidate` is an evaluated proposed module improvement.

3.2.1 Extended Rationale

- **Data Sovereignty:**

  The first design obligation of **MADRE** is to keep the user's data under the user's control. Contemporary AI services often make private life, professional work, organizational knowledge,

and creative material flow through remote systems by default. That default is not neutral. It concentrates power in the companies that own the infrastructure, the logs, the model services, the policy switches, and the billing relation.

MADRE refuses to make surveillance a prerequisite for assistance. Local-first operation is a security boundary, a political boundary, and an engineering discipline. Private conversation, module-owned known material, organizational records, and sensitive work must remain local unless an explicit policy, classification, minimization, and trace path says otherwise.

Internet access is treated as a risk boundary. Prompt injection, credential exposure, external action abuse, provider lock-in, and remote data retention are normal failure modes of cloud-first agentic systems; **MADRE** 's architecture must make those failures harder by default and visible when they cannot be eliminated.

- **Free Software Duty:**

  MADRE is released under `GPL-3.0` because source availability alone is not enough. The project aims to provide usable agentic infrastructure for people and organizations that cannot, or should not, depend on platform owners for every intelligent interface they build.

  The current generation of AI models draws on a broad inheritance of human language, labor, research, art, software, and social knowledge. A public-interest response requires practical, inspectable infrastructure outside exclusive corporate channels.

  MADRE supports developers and creators running useful systems on ordinary machines, inspecting decisions, replacing inference backends and authorized integrations, and sharing improvements without requiring platform permission.

- **Model Agnosticism:**

  No single model, host, vendor, format, or prompt convention should define **MADRE** 's architecture. Models improve, fail, disappear, become expensive, change license terms, and carry different bias and safety profiles.

  MADRE treats inference as a governed `ModelAction`. The runtime may choose different permitted `InferenceProfile` and `InferenceBackend` combinations without letting any model become authority over policy, knowledge, actions, or truth.

- **Resource Discipline:**

  Local-first AI must remain useful on ordinary commercial devices and degrade intelligently when resources are scarce.

  Optional Complex Agent lanes, task decomposition, caching, context minimization, repeated checks where they matter, and durable records make stronger reasoning selective rather than an unlimited default.

- **Knowledge Governance:**

  Agentic systems become dangerous when they blur text, known material, retained interaction, instruction, evidence, and action. A conversation turn is not a verified fact, an `InferenceOutput` is not a source or authority, and an action result is not an instruction.

  A `KnowledgeRecord` carries provenance, sensitivity, intended use, `truthLevel`, and `truthAuthority`; a `LearningCandidate` is an evaluated module-improvement proposal, not ordinary stored knowledge.

- **Distributed Reasoning:**

  Interaction can remain coherent while module-owned agents defer hard work, retrieve permitted material, test hypotheses, revise plans, and learn from evaluated outcomes without treating language models as minds.

  Every agent, `InferenceRecord`, action invocation, `KnowledgeRecord` change, and `LearningCandidate` must remain accountable to an explicit runtime boundary.

3.3 Product Design

3.3.1 Product Scope

MADRE provides a disciplined coordination layer underneath agentic applications. The kernel routes interaction, registers the Default interaction module, records delayed work through `RuntimeJournal`, coordinates bounded context and actions, records `PolicyDecision` outcomes, and exposes recovery semantics.

The primary audience is developers and organizations building private, domain-aware, extensible assistant systems. Applications built on **MADRE** may vary in interface, domain, and policy, but they inherit the same authority model: modules own their agents and known material, model execution is constrained by `ModelBinding`, and policy, trace, action, and recovery boundaries remain runtime-controlled.

3.3.2 What MADRE Is

- A local-first runtime kernel for agentic systems.
- A model-agnostic coordination layer for multiple model families.
- A delayed-reasoning architecture that keeps conversation responsive while deeper work happens through module-local agents and `WorkRecord` lifecycle.
- A policy-aware boundary for context, internal actions, knowledge, learning, and remote transfer.
- A developer foundation for building assistants, domain systems, and workflow-capable applications on top of governed local intelligence.

3.3.3 What MADRE Is Not

- **MADRE** is not a chatbot product. Chat is one possible surface over the kernel, not the kernel's definition.
- **MADRE** is not a prompt wrapper. Prompts are runtime inputs, not an architecture.
- **MADRE** is not a vendor abstraction layer whose purpose is merely to swap API providers.
- **MADRE** is not an unrestricted automation agent. Internal actions, external actions, knowledge-boundary change, and learning require explicit boundaries.
- **MADRE** is not a passive record of interaction history. The product is the governed runtime kernel and its architectural obligations.

3.3.4 Core User Value

MADRE lets an application answer quickly without pretending every answer can be safe, complete, or well-reasoned in the foreground. The Default interaction module can assign a `ModuleMainAgent` with a Complex Agent profile whose foreground lane clarifies intent, uses appropriately classified material, names uncertainty, and triggers deeper work. Module-local reasoning lanes perform work requiring decomposition, evidence, internal actions, verification, or stronger context.

This separation gives users a better contract. They are not forced to choose between a quick hallucination and a slow black box. They get immediate conversation when that is appropriate, delayed reasoning when that is necessary, and inspectable boundaries when the system touches sensitive data or meaningful actions.

3.3.5 Design Principles

- **Local-first, not local-only:** Local execution is the default because privacy, cost, resilience, and agency require a strong local base. Remote services may exist only as explicitly governed integrations with classification, minimization, policy, provenance, and journaled trace.
- **Model-agnostic by contract:** Models are replaceable runtime functions. The kernel must not let any model provider define the system's truth model, policy boundary, knowledge representation, or action authority.
- **Foreground lane, durable reasoning:** A `ModuleMainAgent` may use a Complex Agent foreground lane to preserve conversational flow. Expensive, uncertain, risky, or evidence-heavy work should become a scheduled `WorkRecord` that can be resumed, inspected, verified, and delivered asynchronously.
- **Context is a governed asset:** Context is not just text stuffed into a prompt. It is selected, minimized, classified, sourced, scoped, and revoked according to the purpose of the task and the authority of the actor using it.
- **Autonomy is bounded: MADRE** must support unattended delayed work, but autonomy is legitimate only when the action has an applicable `PolicyDecision`, is recorded through `RuntimeEvent`, is recoverable for its risk level, and is blocked from crossing stronger boundaries without authorization.
- **Learning is evaluated:** Useful systems should improve from use, but a `LearningCandidate` cannot quietly rewrite protected behavior. Saving a `KnowledgeRecord` is not learning; knowledge-boundary and behavior changes must be reviewable, promotable, reversible, and separable.
- **Interfaces do not own the architecture:** The `madre` CLI, local read-first inspection console, graphical UI, speech surfaces, or domain applications are access surfaces. They must expose and operate the kernel without becoming the source of runtime authority.

3.3.6 Product Boundaries

The product boundary covers runtime responsibilities and architecture obligations: local-first operation, model-agnostic inference, module-owned agent coordination, governed context, bounded internal actions, knowledge handling, evaluated learning, runtime traceability, and recovery.

MADRE may expose many application forms: personal assistants, research systems, developer environments, organizational knowledge workers, or domain-specific automation environments. Those applications may differ in interface and policy, but they must inherit the same kernel commitments: local-first operation, replaceable inference, governed context, bounded internal actions, inspectable `ConversationRecord` and `KnowledgeRecord` handling, evaluated learning, journaled runtime history, and recovery.

Operational Scenarios

4.1 Scenario Cards

Scenario / trigger	Module(s)	Runtime objects	Authority boundary	Expected records	Negative path
Foreground continuity / ordinary interaction	Default interaction module	`ModuleMainAgent`, `ContextBundle`, `ConversationRecord`	Answer only from permitted context and without making known material factual by implication.	Request and response `RuntimeEvent`; retained `ConversationRecord` where configured.	Ambiguity, insufficient context, or risk creates clarification, block, or `WorkRecord`, not an unsupported answer.
Delayed research / evidence required	Default interaction module and research module	`ReasoningPlan`, `WorkRecord`, `ContextBundle`, optional `ModelAction`	Routing, data scope, and each action require applicable `PolicyDecision`.	Work lifecycle and evidence references in `RuntimeJournal`; `InferenceRecord` when inference runs.	Unverified or disallowed sources do not become accepted knowledge or authoritative response.
Knowledge handling / material selected for module use	Owning reasoning module	`KnowledgeRecord`; `KnowledgeCandidate` only for pending boundary change	Provenance, scope, sensitivity, intended use, `truthLevel`, `truthAuthority`, correction, and lifecycle metadata govern use.	Knowledge creation/correction/detachment `RuntimeEvent` and related `PolicyDecision`.	Known material with unsuitable truth metadata is not presented as factual authority; saving it is not learning.
Specialized interaction / domain assistance requested	Default interaction module and selected domain module	`ConversationRecord`, `ContextBundle`, module-owned `MADREAgent`, `ModelBinding`	A transferred or delegated scope does not transfer policy or knowledge authority.	Routing event, selected module, binding reference, context reference, and policy outcome.	Domain-specific output remains `InferenceOutput` until explicitly handled; it does not self-route or self-promote.
Workflow execution / reusable behavior invoked	Owning reasoning module	`Workflow`, `AgentRoutine`, `AgentAction`, `WorkRecord`	Each executable action and external crossing is policy-gated with recovery expectation.	Action, work, policy, failure, and recovery `RuntimeEvent` history.	A hidden prompt chain or unauthorized side effect is blocked or authorization-required.
Evaluated improvement / repeated outcome suggests change	Owning reasoning module	`LearningCandidate`, evaluation evidence, optional `KnowledgeRecord` input	Promotion changes module behavior only after declared evaluation and promotion authority.	Candidate creation, evaluation, promotion/rejection, and rollback events.	`InferenceOutput`, conversation retention, or `KnowledgeRecord` storage alone never activates learning.

Developers may add models, internal actions, authorized external actions, projections, context policies, workflows, interfaces, and analysis helpers without rewriting the kernel, provided module ownership, `ModelBinding`, `PolicyDecision`, and `RuntimeJournal` trace obligations remain intact.

Conceptual Architecture

5.1 Formal Requirements

This chapter states architectural requirements at product level.

Requirement	Architectural meaning
Local-first operation	The kernel must be able to run useful workflows on local user-owned infrastructure before optional remote integrations are considered.
Model agnosticism	All model execution enters through `ModelAction`, allowed `ModelBinding`, replaceable `InferenceProfile`, and `InferenceBackend` contracts, never through a vendor-specific core design.
Delayed reasoning	The runtime must separate immediate conversation from deeper work that needs decomposition, evidence, verification, internal actions, or more compute.
Context governance	Context must be selected, minimized, classified, provenanced, scoped, and revocable before it reaches models or internal actions.
Bounded internal actions	Executable behavior must be declared, policy-checked, traceable, and constrained by side effect and recovery rules. Typed runtime-defined internal actions are the canonical low-risk action form.
Record/knowledge isolation	`ConversationRecord`, `KnowledgeRecord`, untrusted source material, `KnowledgeCandidate`, and `LearningCandidate` must not collapse into one undifferentiated prompt store.
Learning control	A `LearningCandidate` is an evaluated proposed improvement and changes active module behavior only through a defined promotion boundary; storing a `KnowledgeRecord` is not learning.
Inspectable execution	The CLI, local developer console, `RuntimeJournal`, and read models must make runtime behavior understandable without bypassing policy.

5.2 High-Level Architecture

MADRE is organized around a deterministic runtime kernel that coordinates governed reasoning modules through responsibility boundaries rather than through a fixed topology.

The central architectural rule is simple: the model is never the system. Models are invoked through `ModelAction` under context, policy, action, knowledge, and trace constraints. Each concrete `MADREAgent` is owned by one `ReasoningModule` and constrained by `ModelBinding`. Agents may be short-lived, persistent, specialized, conservative, speculative, or workflow-bound, but they do not own the kernel or carry module knowledge across ownership boundaries.

The kernel coordinates modules; it does not become the reasoning actor for every task. System-level reasoning belongs in governed modules. Kernel registration assigns one compatible `ReasoningModule` the Default interaction module role for incoming interaction, continuity, clarification, planning, delegation, and delayed-work decisions. Within that module, one `MADREAgent` holds the `ModuleMainAgent` role; a `ComplexAgent` profile may fulfill it but is not synonymous with it.

5.2.1 Conceptual Components

Component	Responsibility
Access surfaces	Present the system through CLI, local read-first inspection console, graphical UI, speech, or domain applications without becoming the source of runtime authority.
Default interaction module	A kernel-assigned `ReasoningModule` role that maintains interaction continuity, clarifies intent, creates reasoning plans, delegates to modules, and decides when deeper work is needed.
Runtime kernel	Coordinates module registration, routing, scheduling, resource limits, policies, action registries, durable records, recovery, and lifecycle.
Reasoning modules	Execute bounded reasoning domains through module-owned agents, workflows, routines, actions, records, policies, known-material boundaries, and allowed model/inference scope.
Context and knowledge	Provides scoped context bundles and `KnowledgeRecord` objects with provenance, scope, sensitivity, intended use, truth metadata, and correction paths.
Local Model Runtime Boundary	Product-level boundary for local or explicitly authorized inference through `ModelBinding`, `InferenceProfile`, `InferenceBackend`, `InferenceRecord`, and `InferenceOutput`.
Internal Action Registry	Exposes typed, bounded internal actions with declared inputs, outputs, side-effect class, policy requirements, trace evidence, and recovery expectations. External actions are exceptional risk boundaries, not the default execution path.
Learning and evolution	Evaluates `LearningCandidate` objects and promotes only authorized module improvements; known-material storage is separate.
Observability and recovery	Gives developers and authorized users a read-first view of `WorkRecord`, `RuntimeEvent`, `PolicyDecision`, failures, and rollback options through `RuntimeJournal`.

5.2.2 Responsibility Layer Diagram

Figure 5.1 presents the primary responsibility structure of **MADRE** . The diagram is not a sequential request pipeline. It describes the ownership boundaries that constrain all runtime behavior.

The **MADRE** runtime kernel is the deterministic system coordinator. It owns access-surface entry, module registration, Default interaction module assignment, module routing, scheduling, resource assignment, global policy boundaries, `RuntimeJournal` coordination, durable runtime state, and lifecycle boundaries. The kernel does not become the reasoning actor for ordinary tasks. It selects, activates, limits, and observes reasoning modules.

Reasoning modules are the runtime units that perform domain-bounded reasoning. The module assigned the Default interaction module role is the first receiver for ordinary user interaction because its registered capability covers interaction continuity, clarification, general planning, and delegation. Other modules may specialize in research, knowledge handling, code, media, resource optimization, or domain-specific work. A module may answer quickly, create delayed work, invoke internal actions, or evaluate outcomes when its local policy and assigned resources allow it.

Executable behavior is not exposed as arbitrary external tools. The normal execution path uses an Internal Action Registry composed of typed runtime-defined functions with declared inputs, outputs, side-effect class, policy requirements, `RuntimeEvent` trace obligations, and recovery expectations. The first reference implementation may implement these actions on a JVM runtime, but JVM binding is not an architecture-level authority. External services, host commands, provider-backed tools, remote model services, or arbitrary plugin surfaces are not part of the default path and require stronger authority boundaries.

The Local Model Runtime Boundary is useful product-level phrasing for the point below the module and action layer where allowed inference executes. It is realized through `ModelBinding`, `InferenceProfile`, `InferenceBackend`, `InferenceRecord`, and `InferenceOutput`; it is not a chatbot server or an authority. Generated material returns to the module as `InferenceOutput` and is not truth, policy, memory, knowledge, behavior, routing, authority, or a candidate by itself.

5.2.3 Concurrent Agent Lanes

MADRE separates immediate interaction from deeper reasoning, but this separation is not modeled as two global subsystems. It is a possible execution pattern for a module-owned `MADREAgent`.

A `MADREAgent` with a `ComplexAgent` profile may maintain three concurrent lanes. The foreground lane preserves conversational continuity, answers only when safe known context is sufficient, asks for clarification when intent or risk is unclear, and creates deferred work when more evidence is required. The reasoning lane performs planning, decomposition, evidence gathering, action invocation, verification, and result construction under module authority. The reflection lane reviews recent work, observes delayed outcomes, detects failure patterns, and proposes evaluated `LearningCandidate` objects.

This pattern may exist in any reasoning module. The Default interaction module may use it for general interaction and delegation. A research module may use it for evidence comparison and verification. A knowledge module may use it for extraction, provenance checks, correction, and deletion semantics. The kernel schedules and constrains these lanes through module routing, resource limits, durable state, policy, and `RuntimeJournal`, but the reasoning behavior itself belongs to module-local agents.

5.3 Runtime Data Flow

Runtime data flow in MADRE is concurrent, governed, and module-centered. A local request enters through an access surface and is submitted to the kernel. The kernel records the request, applies the relevant entry policy, and routes it to the Default interaction module by registration or to another registered module when routing evidence supports that decision.

Within the selected module, its `ModuleMainAgent` may produce an immediate foreground response, create or continue delayed reasoning work, invoke internal actions, request inference through an allowed `ModelAction`, and reflect on recent outcomes. These activities are coordinated inside a module-owned agent and constrained by module policy, kernel policy, `ModelBinding`, resource assignment, durable state, `RuntimeJournal`, and recovery rules.

The following subsections describe the canonical request narrative, the governed model/action sequence, and the data-authority rules that prevent user input, source material, `InferenceOutput`, action

results, or learning candidates from promoting themselves into authority-bearing state.

5.3.1 Canonical Request Narrative

The canonical user request begins when an access surface submits a local request to the kernel. The kernel does not interpret the request as a `KnowledgeRecord`, factual authority, or policy. It records a `RuntimeEvent`, applies entry constraints, and routes the work to the Default interaction module unless another module is already selected by session state, policy, or module registry evidence.

The receiving module evaluates the request through its module-local `ModuleMainAgent`. If a configured foreground lane has enough safe local context, it may produce an immediate answer. If intent, scope, or risk is unclear, it asks for clarification. If the request requires evidence, stronger reasoning, internal actions, model inference, or broader context, the reasoning lane creates or continues a `WorkRecord`. Reflection may run after the response or after delayed outcomes to propose a `LearningCandidate`. Knowledge-boundary change is separately represented as a `KnowledgeCandidate`.

Every significant step emits a `RuntimeEvent` through `RuntimeJournal`. Context bundles, actions, `InferenceRecord` and `InferenceOutput` objects, `PolicyDecision` outcomes, failures, correction, rollback, invalidation, supersession, and deletion/detachment remain inspectable through read-first surfaces.

5.3.2 Governed Model And Action Sequence

When module work requires executable behavior, the module invokes an internal typed action. The default action mechanism is local, typed, runtime-defined, policy-gated, and traceable. It is not an open plugin socket and not a network-exposed external action protocol.

A `ModelAction` is an `AgentAction` that uses inference. Before it runs, the module supplies a governed `ContextBundle`, an allowed `ModelBinding` and `InferenceProfile`, and the applicable `PolicyDecision`. An `InferenceBackend` executes inference without allowing a provider, template, or runtime format to define MADRE 's authority model. One execution is captured as an `InferenceRecord`; its generated material is an `InferenceOutput`.

Action results may be observations. `InferenceOutput` is generated material and is not a candidate by itself. It may inform a next reasoning step or undergo explicit handling into another record, but it does not authorize actions, create policy, become memory or knowledge, change routing or behavior, or promote learning. Each such transition requires its relevant authority boundary, verification path, `RuntimeEvent` record, and recovery expectation.

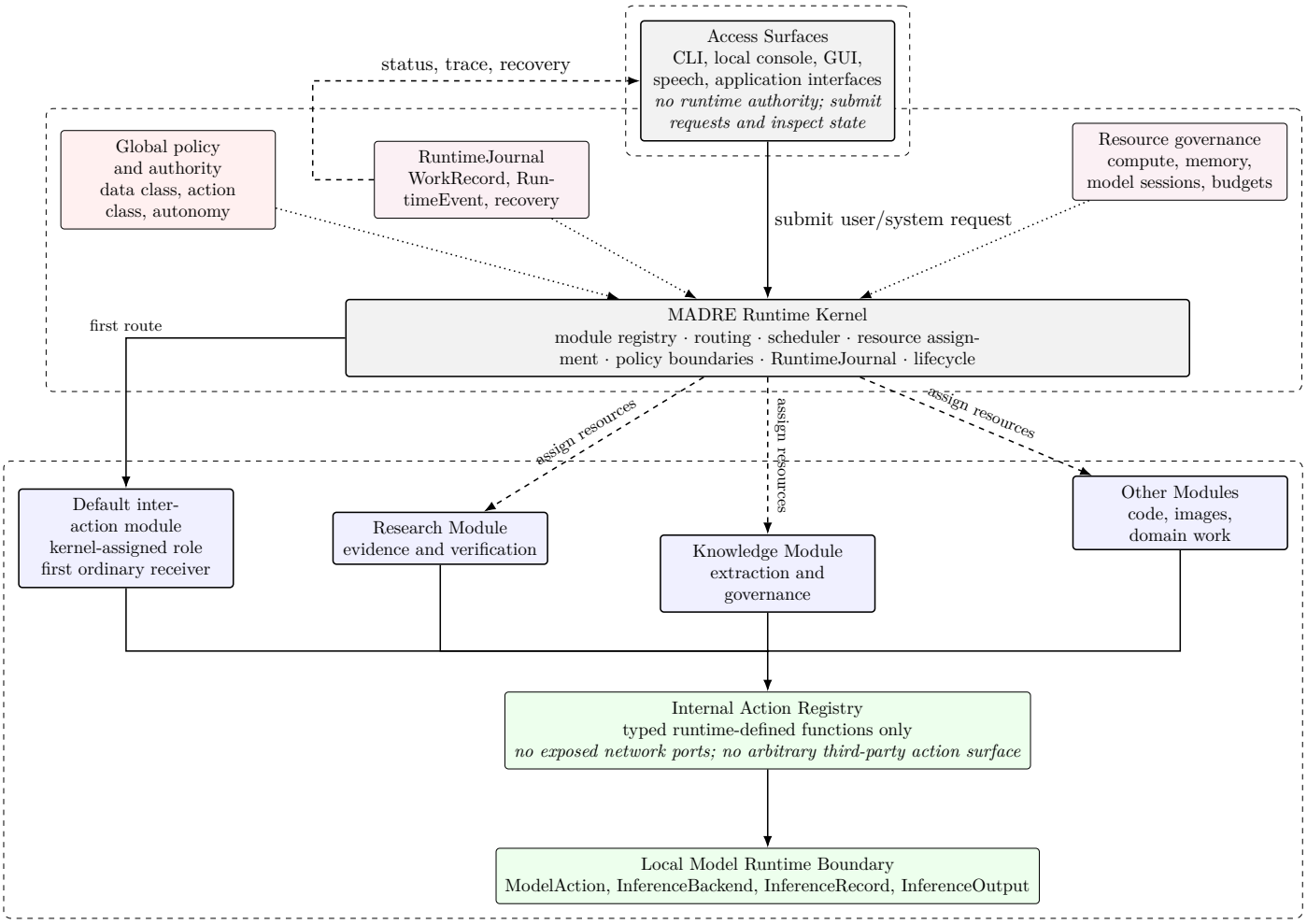


Figure 5.1: MADRE responsibility layers: kernel coordination, Default interaction module assignment, runtime journal, actions, and inference.

Figure 5.1 defines the top-level responsibility model. Access surfaces are intentionally outside runtime authority: they submit requests and inspect state, but they do not own policy, knowledge, routing, scheduling, or action-authority decisions. The kernel is the only system-level coordinator. It registers modules, assigns the Default interaction module, routes requests, assigns resources, applies global policy boundaries, records through `RuntimeJournal`, and maintains durable lifecycle state.

The module layer is the reasoning layer. The Default interaction module is shown first because ordinary interaction is routed there by registration, not because it is outside the module system. Research, knowledge, code, media, or domain modules are peers selected when the registry, policies, and current reasoning plan indicate a better fit. Internal actions and inference appear below the module layer because they are invoked by authorized module behavior, not by access surfaces or `InferenceOutput` directly.

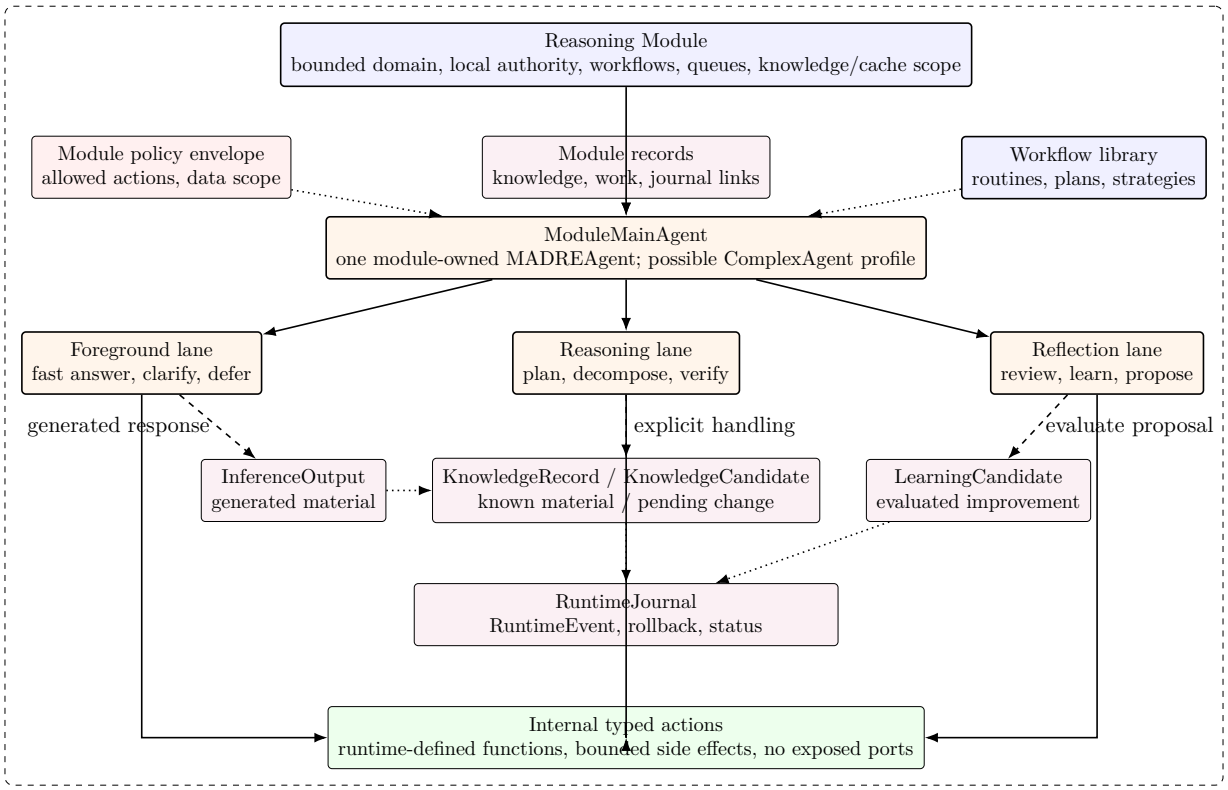


Figure 5.2: Reasoning module internal structure with its ModuleMainAgent possibly using a ComplexAgent lane profile.

Figure 5.2 expands the internal structure of a reasoning module. A module owns a bounded domain, local policy envelope, records, workflow library, knowledge boundary, and concrete `MADREAgent` objects. One agent holds the `ModuleMainAgent` role. A `ComplexAgent` profile may be assigned when concurrent lanes are useful, but it is not that role itself.

The foreground lane, reasoning lane, and reflection lane may operate concurrently. The foreground lane protects user continuity. The reasoning lane protects evidence, decomposition, verification, and stronger work. The reflection lane protects adaptation by evaluating a proposed `LearningCandidate` rather than silently rewriting runtime behavior. These lanes do not increase agent authority. They remain constrained by module policy, kernel policy, assigned resources, trace requirements, and promotion boundaries.

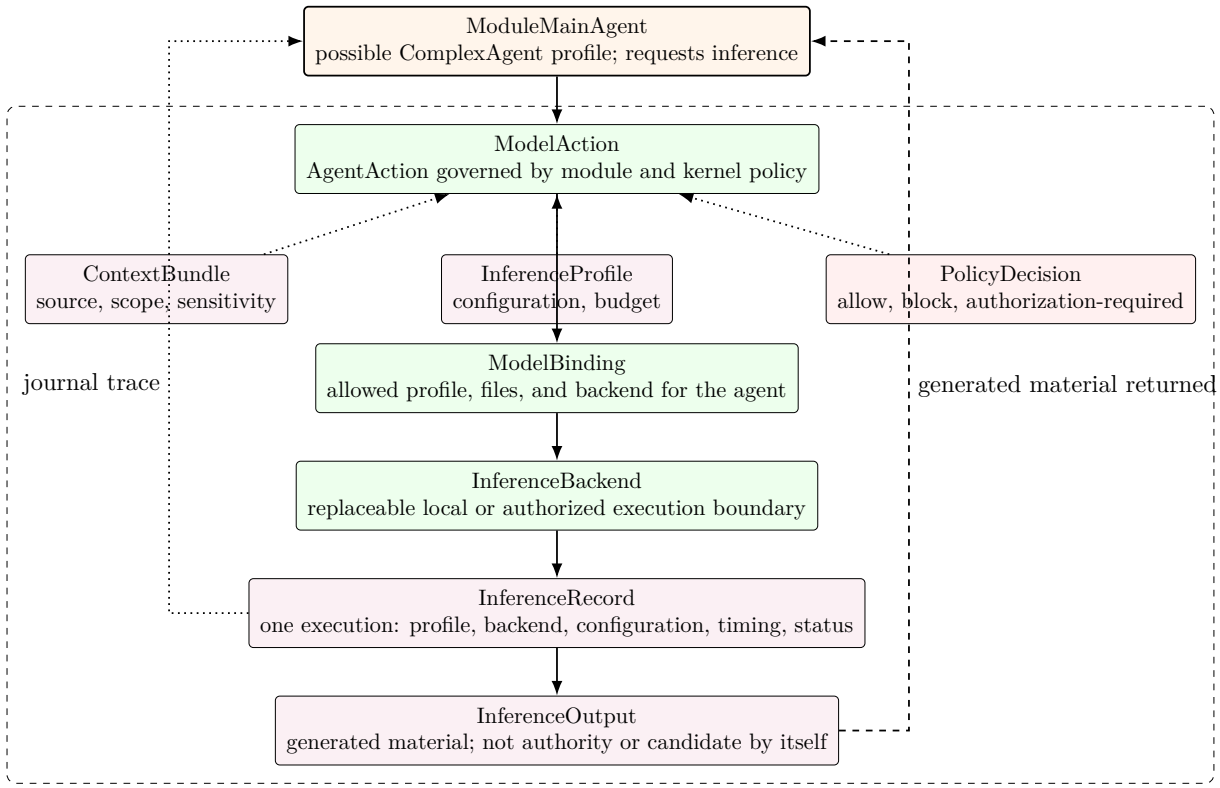


Figure 5.3: ModelAction inference path from governed execution through InferenceRecord to non-authoritative InferenceOutput.

Figure 5.3 describes inference as the governed `ModelAction` path rather than as an external server call or chat-completion product boundary. A module-owned `MADREAgent`, including one serving as `ModuleMainAgent`, can request generation, critique, classification, or verification only under its allowed `ModelBinding` and an applicable `PolicyDecision`.

The `InferenceProfile` captures execution configuration, and the replaceable `InferenceBackend` executes the permitted inference. Every execution creates an `InferenceRecord`; the generated material attached to that execution is `InferenceOutput`. The Local Model Runtime Boundary remains useful as product wording for this controlled local or authorized execution boundary, not as a competing runtime class.

`InferenceOutput` is not truth, policy, memory, knowledge, behavior, routing, authority, or a candidate by itself. Explicit module handling can retain it as `ConversationRecord`, save appropriately classified material as `KnowledgeRecord`, evaluate it toward a `LearningCandidate`, or discard it, with the transition recorded in `RuntimeJournal`.

5.3.3 Data And Authority Rules

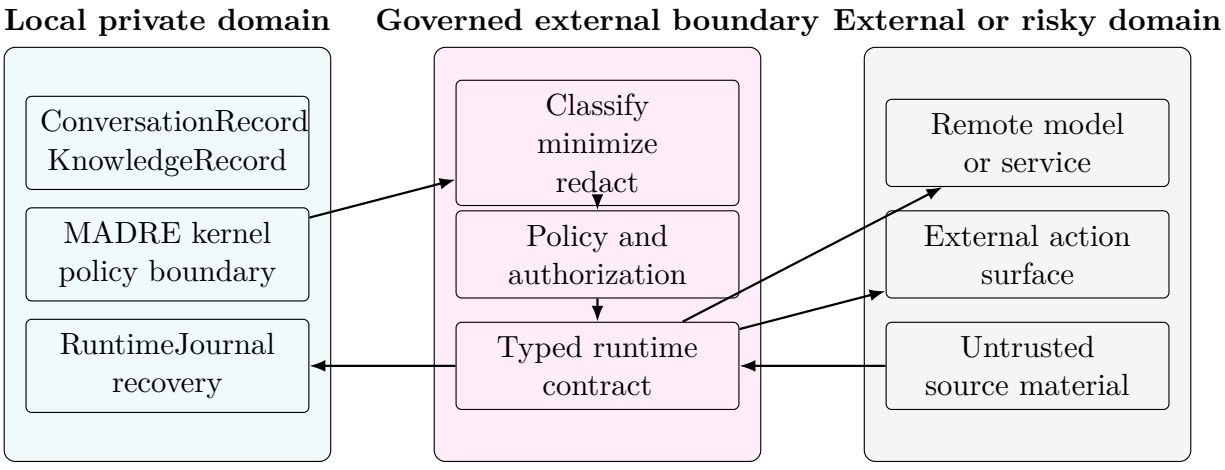
- User input enters as `ConversationRecord` material when retained, not as policy, knowledge, truth, action authority, or model authority.
- Retrieved, extracted, or saved material becomes `KnowledgeRecord` only with provenance, scope, sensitivity, intended use, `truthLevel`, `truthAuthority`, correction, and lifecycle metadata; it is not factual truth by default.

- Context bundles are constructed, minimized, classified, scoped, and revocable before they reach an agent lane, internal action, or `ModelAction`.
- `InferenceOutput` is generated material from an `InferenceRecord`; it does not become an answer, policy decision, memory, knowledge, behavior, routing, authority, or candidate by itself.
- Internal action results enter as bounded observations. They become relied-on evidence only according to the action contract, verification path, and `RuntimeEvent` trace.
- External action surfaces, host commands, remote services, arbitrary filesystem mutation, credentials, and irreversible effects require stronger authority boundaries than ordinary internal actions.
- A `KnowledgeCandidate` represents only pending knowledge-boundary change; a `LearningCandidate` represents evaluated proposed module improvement and cannot directly rewrite active runtime behavior.

5.3.4 Trust Boundary Diagram

The trust boundary diagram distinguishes the ordinary local governed domain from optional external or risky domains. The default execution path remains inside the local governed domain: access surfaces, kernel coordination, reasoning modules, internal typed actions, local model runtime, `RuntimeJournal`, and recovery.

The external boundary exists for exceptional external actions: remote services, provider-backed models, host-command execution, arbitrary filesystem mutation, network access, untrusted source acquisition, or third-party integrations. These are not ordinary internal actions. They require classification, minimization, explicit policy authorization, recorded trace, and a recovery or blocking expectation appropriate to their risk class.



5.3.5 Access Surfaces

MADRE exposes access surfaces without allowing those surfaces to define the kernel. The `madre` CLI submits local requests and exposes status or result evidence. A local read-first inspection console may expose sessions, queued work, context bundles, policy outcomes, model evidence, internal action evidence, `KnowledgeCandidate`, `LearningCandidate`, `RuntimeJournal` traces, recovery, and runtime state. These surfaces operate under the same policy and authority model as every other runtime interaction.

The inspection console is not a privileged mutation path. Its transport is intentionally not fixed at architecture level. If a later implementation uses HTTP, sockets, embedded UI, or another mechanism, that choice must be justified by deployment and security decisions and must preserve local-first, read-first, policy-governed behavior.

Graphical, speech, product-specific, or integration surfaces remain access surfaces over the runtime rather than privileged paths around it.

Safety And Evolution Policy

6.1 Runtime Safety

MADRE is intended to run useful delayed work while the user is not constantly present. That requirement makes safety an architectural property, not a documentation afterthought. The system must be able to explain what it did, why it did it, what data it used, what boundary allowed it, and how the result can be corrected or rolled back when the action is reversible.

6.1.1 Policy Before Action

The kernel must not expose executable behavior merely because a model can request it. Internal actions, external calls, knowledge-boundary changes, workflow execution, and learning promotion need an explicit `PolicyDecision` before execution. The safe path should be the normal path, and safe autonomous improvement should not require constant user interruption. Unsafe paths should be technically isolated, traceable, recoverable where possible, and visible to the responsible developer or operator through `RuntimeJournal`.

Typed runtime-defined internal actions represent the canonical bounded action form. They may be allowed only through a policy-gated action boundary. Remote action, host commands, arbitrary filesystem access, mutation actions, provider-backed integrations, active learning promotion, and workflow mutation require stronger authority boundaries than bounded internal action execution.

6.1.2 Autonomy Levels

Level	Meaning
Observation	Read local runtime state, existing knowledge, and safe metadata without mutation.
Draft creation	Produce summaries, candidate records, or proposed changes without promoting them into active state.
Bounded mutation	Modify authorized local state only when the action is logged, scoped, reversible where possible, and policy-permitted.
External action	Contact remote systems, use external services, or disclose data only after classification, minimization, authorization, provenance, and recorded trace.
Critical action	Credentials, identity, legal, financial, destructive, irreversible, or executable-code actions require a stronger human authorization path.

6.2 Evolution Control

6.2.1 Evolution Rules

- Runtime learning is represented by evaluated `LearningCandidate` proposals, not only by manual feedback collection.
- Learning strategies must declare what they are allowed to observe, propose, mutate, verify, and promote.
- A `LearningCandidate` cannot silently modify protected behavior.
- Learning proposals begin with source, evaluation evidence, scope, and rollback information before they are promoted.
- A `KnowledgeCandidate` is used only for pending knowledge-boundary change and requires provenance, truth metadata, correction paths, and recovery options.
- Saving a `KnowledgeRecord` is not learning by itself.
- Workflow and routing improvements may become default behavior through developer-defined checks, evaluation gates, and rollback policies, without requiring user authorization for every routine improvement.
- Model adaptation, fine-tuning, or weight updates are advanced model-evolution actions governed by stronger model-adaptation authority.
- Critical learning failures must be recoverable through snapshots, versioned state, VCS-backed records, or an equivalent rollback mechanism.

This policy keeps **MADRE** capable of evolving through conscious, checked learning without reducing improvement to constant user authorization. The final user should be interrupted as little as possible. Routine learning should be governed by developer-defined strategies, evidence, tests, promotion gates, `RuntimeJournal` traces, and rollback mechanisms; user authorization is reserved for boundaries where risk, privacy, identity, external action, or policy explicitly requires it.

D-011 fixes the controlled-promotion rule: `KnowledgeCandidate` denotes pending knowledge-boundary change and `LearningCandidate` denotes evaluated proposed module improvement. Neither candidate creation nor `KnowledgeRecord` storage silently rewrites protected behavior.

System Engineering Baseline

7.1 Engineering Problem

MADRE addresses the engineering problem of coordinating model inference without allowing the model to become the system. The runtime must support foreground continuity, delayed reasoning, governed context, bounded internal actions, known material, learning, runtime traceability, and recovery while preserving local ownership of data and operational control.

The core difficulty is not only model accuracy. A useful local assistant must decide when an answer is safe in the foreground lane, when work must be delayed, which context may be exposed, which internal action may be invoked, which result requires verification, and how a mistaken or unsafe outcome can be inspected, corrected, or rolled back.

7.2 Stakeholders And Concerns

MADRE serves users who need private assistance, developers who need an extensible runtime, organizations that need governed local knowledge, and maintainers who must keep the system auditable and evolvable. Each stakeholder depends on the same architectural guarantees: model independence, local-first operation, explicit authority boundaries, understandable execution, and recoverable state.

The principal concerns are privacy, cost, autonomy control, model substitution, context minimization, data provenance, action safety, knowledge correction, learning control, operational continuity, and evidence for system behavior. A runtime that cannot expose those concerns as inspectable state cannot satisfy **MADRE** 's product objective, even if its conversational output appears fluent.

7.3 Requirement Domains

MADRE 's functional baseline is organized around runtime domains. The foreground lane domain preserves conversational flow, asks for clarification, and creates deeper work when the answer requires evidence or internal actions. The delayed reasoning domain records work durably, schedules it, verifies results, and returns outcomes without erasing the foreground interaction.

The context domain selects, minimizes, classifies, scopes, and revokes information before it reaches models or internal actions. The inference domain uses `ModelAction`, permitted `ModelBinding`, `InferenceProfile`, and replaceable `InferenceBackend` contracts. The internal action domain exposes declared side effects under `PolicyDecision` and `RuntimeEvent` recording. The knowledge domain separates `ConversationRecord`, `KnowledgeRecord`, pending `KnowledgeCandidate`, untrusted source material, and evaluated `LearningCandidate`.

The learning domain keeps improvement proposals quarantined until evidence, evaluation, promotion, and rollback rules allow activation. The observability domain exposes sessions, `WorkRecord` lifecycle, context bundles, `InferenceRecord`, actions, `PolicyDecision`, failures, metrics, and recovery paths. The deployment domain keeps ordinary local operation viable before optional remote integrations are considered.

7.4 Traceability Surface

Each significant product claim in **MADRE** must be visible as system behavior. A claim such as local-first operation corresponds to local execution paths, storage boundaries, remote-transfer controls, and inspection evidence. A claim such as model agnosticism corresponds to replaceable `InferenceProfile` and `InferenceBackend` contracts and tests that prevent provider-specific assumptions from entering the kernel.

The same trace applies to safety claims. Policy before action must appear as an enforceable boundary around internal actions, external calls, knowledge-boundary change, workflow execution, and learning promotion. Governed context must appear as provenance, classification, minimization, scope, revocation, and journaled trace. Delayed reasoning must appear as `WorkRecord`, `RuntimeEvent` transitions, verification, result delivery, and recovery.

7.5 Quality Attributes

MADRE prioritizes data sovereignty, privacy, resource discipline, responsiveness, reliability, recoverability, maintainability, portability, auditability, and resistance to hallucination-driven action. These attributes are not independent decorations; they define whether the runtime fulfills its purpose.

Responsiveness is measured by preserving useful foreground conversation while heavier work is delayed. Reliability is measured by durable records, repeatable status, bounded retries, and visible failures. Maintainability is measured by the ability to replace models, internal actions, workflows, and interfaces without rewriting the kernel. Auditability is measured by the ability to explain what context, `InferenceRecord`, action, `PolicyDecision`, and verification path produced an outcome through `RuntimeJournal`.

7.6 Risk And Policy Boundaries

MADRE treats prompt injection, data leakage, excessive agency, unsafe action execution, poisoned knowledge, provider dependency, remote retention, unbounded consumption, credential exposure, and rollback gaps as architectural risks. The runtime must assume that `InferenceOutput` can be persuasive, wrong, incomplete, unsafe, or maliciously influenced by context.

Risk control is expressed through policy boundaries. Data classification governs what may be retrieved, stored, disclosed, or sent to a model. Autonomy level governs whether a result may remain a draft, update local state, invoke an internal action, contact an external service, or require stronger authorization. Recovery expectation governs whether the action must be reversible, snapshot-backed, compensatable, or blocked.

Requirements Baseline

8.1 Requirement Taxonomy

Prefix	Requirement family
FR	Functional behavior of foreground-lane conversation, delayed reasoning, context construction, inference, internal actions, knowledge, learning, runtime trace, and recovery.
NFR	Quality attributes such as responsiveness, reliability, recoverability, maintainability, portability, resource discipline, and auditability.
SEC	Security and trust controls, including prompt injection resistance, credential protection, action authority, and remote-transfer boundaries.
DATA	Data, knowledge, provenance, retention, deletion, correction, revocation, and quarantine requirements.
AI	`ModelAction`, model substitution, `InferenceOutput` handling, and verification requirements.
UX	User-visible states, clarification, authorization, progress, failure, correction, and result-delivery requirements.
OPS	Local deployment, storage, backup, diagnostics, migration, and operational recovery requirements.

8.2 Functional Requirements

ID	Requirement	Acceptance criteria	Verification
FR-001	The runtime shall preserve foreground-lane conversation while delaying evidence-heavy, risky, action-dependent, or broad-context work.	Immediate answers stay within safe known context; stronger work is recorded as delayed work.	Triage scenarios; D-001, D-004.
FR-002	The runtime shall represent delayed work as `WorkRecord` objects with inspectable status and recovery path.	Work exposes queued, running, blocked, failed, completed, and recovered states with `RuntimeEvent` history in `RuntimeJournal`.	Work-lifecycle and recovery scenarios; D-001, D-003, D-008.
FR-003	The runtime shall build context bundles from sourced, classified, minimized, scoped, provenanced, and revocable material.	Context is validated or rejected before `ModelAction` or internal action use.	Context-governance scenarios; D-001, D-005, D-007.
FR-004	The runtime shall execute model-backed work through `ModelAction` constrained by `ModelBinding` and replaceable `InferenceProfile`/`InferenceBackend` contracts.	Every execution produces an inspectable `InferenceRecord` and keeps its `InferenceOutput` non-authoritative.	Inference substitution and output-boundary scenarios; D-001, D-006.
FR-005	The runtime shall expose executable behavior only as bounded internal actions or explicitly authorized external actions.	Internal actions remain policy-gated, while disallowed action requests are blocked or authorization-gated.	Action policy scenarios; D-007, D-009.
FR-006	The runtime shall keep `KnowledgeCandidate` and `LearningCandidate` semantics separate and inactive unless their respective promotion rules activate them.	Saving a `KnowledgeRecord` does not activate learning; evaluated behavior, routing, workflow, or inference-selection improvement remains a `LearningCandidate` until promoted.	Knowledge-boundary and learning-promotion scenarios; D-011.
FR-007	The runtime shall expose local request entry and read-first inspection.	Local CLI request/status interaction and local console inspection expose system state without bypassing runtime authority.	Local access and inspection scenarios; D-010.
FR-008	The runtime shall represent recovery outcomes explicitly.	Correction, rollback, deletion/detachment, invalidation, and supersession are distinguishable and auditable.	Recovery readout scenarios; D-003, D-008.

8.3 Cross-Cutting Requirements

ID	Requirement	Rationale	Verification
NFR-001	Local useful operation shall remain possible without mandatory remote service dependency.	Data sovereignty, cost control, and resilience.	Local execution evidence; D-001, D-002, D-006.
NFR-002	Runtime state changes shall be inspectable through local read models.	Auditability and recoverability.	Inspection review; D-003, D-010.
NFR-003	Delayed work shall be bounded by status, resource, and failure discipline.	Prevents unbounded cost, compute, storage, and queue growth.	Resource/status scenarios; D-003, D-007, D-010.
SEC-001	Prompt or source material shall not become instruction, policy, or authority without explicit classification and boundary checks.	Source-as-instruction injection risk.	Adversarial context scenario; D-004, D-005, D-007.
SEC-002	A `PolicyDecision` shall explicitly represent allow, block, or authorization-required before action execution or promotion and be linked to `RuntimeEvent`.	Prevents hidden authority decisions and excessive agency.	Policy matrix review; D-007, D-009, D-011.
SEC-003	External, destructive, credential, legal, financial, executable, or irreversible effects shall be blocked or separately authorized.	Preserves recoverability and explicit authority over critical action.	Critical-action negative paths; D-007, D-008, D-009.
DATA-001	A `KnowledgeRecord` shall carry provenance, scope, sensitivity, intended use, `truthLevel`, `truthAuthority`, correction, and lifecycle metadata; it is not factual truth by default.	Knowledge governance and user control.	Non-authoritative known-material, correction, and revocation scenarios; D-005, D-008, D-011.

ID	Requirement	Rationale	Verification
DATA-002	Active state and accountable `RuntimeEvent` history shall remain distinguishable.	Allows deletion/detachment without losing required trace history.	Recovery inspection scenarios; D-003, D-008.
AI-001	`InferenceOutput` shall remain generated material from an `InferenceRecord`, not truth, policy, memory, knowledge, behavior, routing, authority, or a candidate by itself.	Hallucination and false-authority risk.	Output non-authority and non-candidate scenarios; D-001, D-006.
AI-002	Models and `InferenceOutput` shall not create or widen `PolicyDecision`, authorize actions, establish knowledge truth, alter behavior or routing, or activate learning.	Keeps runtime authority outside generated material.	Model-overreach and promotion-control scenarios; D-006, D-007, D-011.
UX-001	Users shall be able to distinguish immediate answers, queued work, running work, blocked work, failed work, completed work, and authorization requests.	Continuity and user control.	Interaction-state review; D-004, D-010, D-011.
OPS-001	Local access surfaces shall submit requests and inspect `RuntimeJournal` status and trace without becoming runtime authority.	Supports operability while preserving governance.	Local access and inspection scenario; D-010.

Traceability Model

9.1 Traceability Method

Traceability in **MADRE** links the product claim, stakeholder concern, requirement, architecture obligation, validation method, and evidence status. The purpose is to keep evaluation anchored to the system thesis: local-first operation, model agnosticism, delayed reasoning, governed context, bounded internal actions, evaluated learning promotion, runtime traceability, and recovery.

9.2 Architecture Traceability Matrix

Claim	Concern	Requirement	Decision trace	Evidence family
Local-first operation	Data sovereignty, cost, resilience	NFR-001	D-001, D-002, D-010	Local request, local status, local journal trace, no mandatory remote service.
Model agnosticism	Provider independence	FR-004, AI-001	D-006	Replaceable `InferenceBackend`/`Infer` substitution plus `InferenceOutput` non-authority scenario.
Delayed reasoning	Responsiveness and quality	FR-001, FR-002	D-001, D-003, D-004	Foreground triage plus durable delayed-work lifecycle.
Governed context and known material	Privacy and correctness	FR-003, DATA-001	D-005, D-007	Context acceptance, source-as-data, non-authoritative `KnowledgeRecord` use, truth-metadata correction, and revocation scenario.

Claim	Concern	Requirement	Decision trace	Evidence family
Bounded internal actions	Safety and recoverability	FR-005, SEC-001	D-007, D-009	Policy-gated internal action and blocked unsafe external action attempts.
Learning promotion control	Controlled evolution	FR-006	D-011	`LearningCandidate` evaluation and inspection without active promotion; `KnowledgeRecord` save is not treated as learning.
Runtime trace and recovery	Accountability and correction	FR-008, NFR-002, DATA-002	D-003, D-008, D-010	`RuntimeJournal`/`Runtime` status, correction, invalidation, supersession, rollback, and deletion/detachment trace.

9.3 Integrated Architecture Trace

Step	Required behavior	Decision trace	Negative evidence
Local entry	Request enters through local access surface.	D-001, D-010	Internal-only execution is not enough to prove inspection.
Foreground triage	Foreground answers only safe known context, clarifies, blocks, or creates delayed work.	D-004, D-007	Evidence-heavy request is not answered as if already known.
Durable work	Delayed work is represented by `WorkRecord` with `RuntimeEvent` status history.	D-003	Work is not lost or silently overwritten.

Step	Required behavior	Decision trace	Negative evidence
Governed context	Context is sourced, classified, minimized, scoped, provenanced, revocable, and linked to journal trace.	D-005, D-007	Sensitive, unrelated, revoked, or incorrectly truth-classified material is excluded or blocked.
Inference boundary	`ModelAction` uses `InferenceBackend`, creates `InferenceRecord`, and returns non-authoritative `InferenceOutput`.	D-006	Generated material does not become authority or a candidate by itself.
Bounded internal action	Typed runtime-defined internal actions are policy-gated and recorded.	D-009	Remote, host-command, filesystem, or mutation actions require stronger authority boundaries.
Knowledge and learning handling	`KnowledgeCandidate` covers pending knowledge-boundary change; `LearningCandidate` covers evaluated improvement.	D-011	`KnowledgeRecord` storage does not activate learning and candidates cannot promote themselves.
Inspection and recovery	Status, trace, failure, correction, rollback, invalidation, supersession, and deletion/detachment are visible through `RuntimeJournal`.	D-008, D-010	Active state changes do not erase accountable history.

Design Decision Traceability

Design decisions D-001 through D-011 provide traceability from product requirements to architecture obligations and validation evidence. They identify accepted product architecture decisions for governed delayed reasoning, durable work, context, inference, policy, recovery, bounded internal actions, local access, knowledge-boundary control, and evaluated learning promotion.

Decision	Product-level effect	Architecture boundary
D-001	Defines source-backed delayed-answer behavior as a core expression of governed runtime value.	Conversation remains connected to durable work and evidence.
D-002	Establishes behavior-first architectural validation as part of the product's engineering discipline.	Observable behavior anchors architecture claims.
D-003	Requires `WorkRecord` lifecycle and projected inspection state.	`RuntimeJournal` and append-only `RuntimeEvent` preserve local traceability.
D-004	Requires conservative foreground triage.	Unsupported answers become clarification, policy block, or delayed work.
D-005	Requires governed context acceptance and rejection.	Context is sourced, classified, minimized, scoped, provenanced, and revocable.
D-006	Requires `ModelAction`, `InferenceBackend`, `InferenceRecord`, and non-authoritative `InferenceOutput`.	Models remain runtime functions, not authorities or candidate creators by generation alone.
D-007	Requires explicit `PolicyDecision` before action execution.	Action execution follows policy rather than model instruction.
D-008	Requires recovery as a recorded semantic transition.	Correction, rollback, invalidation, supersession, deletion, and detachment have distinguishable `RuntimeEvent` meaning.

Decision	Product-level effect	Architecture boundary
D-009	Defines bounded internal actions as the low-risk executable reference.	Action execution is policy-gated, traceable, and recoverability-aware.
D-010	Defines CLI request/status interaction and local read-first console inspection as canonical local access surfaces.	Access surfaces expose the kernel without becoming runtime authority.
D-011	Distinguishes pending `KnowledgeCandidate` boundary changes from evaluated `LearningCandidate` module improvements.	Known-material storage is not learning; proposed evolution remains inspectable until promoted by authority.

Architecture Viewpoints

11.1 Viewpoint Catalogue

Viewpoint	Primary concern	MADRE content
System context	Runtime boundary and environment	User, local device, filesystem, operating system, local models, optional external integrations, internal actions, and untrusted sources.
Logical	Responsibility separation	Kernel, Default interaction module, module-owned agents, scheduler, context/knowledge, Local Model Runtime Boundary, Internal Action Registry, `PolicyDecision`, `RuntimeJournal`, learning, and recovery.
Runtime/process	Work lifecycle	User turn, triage, `WorkRecord`, context bundle, scheduling, `InferenceRecord`, result delivery, `RuntimeEvent`, and recovery.
Data/knowledge	Data ownership and provenance	`ConversationRecord`, `KnowledgeRecord`, `KnowledgeCandidate`, `LearningCandidate`, truth metadata, retention, correction, and deletion.
Security/trust	Boundary enforcement	Local domain, governed external boundary, external/risky domain, prompt injection, remote transfer, credential exposure, and action side effects.

Viewpoint	Primary concern	MADRE content
Deployment	Local operation	Ordinary device profile, local storage, model availability, backup, migration, diagnostics, and optional remote configuration.
Extension	Developer evolution	`ModelBinding`, `InferenceProfile`, `InferenceBackend`, action contracts, workflows, projections, context policies, and interface surfaces.
UX	User-visible control	Conversation, clarification, queue state, progress, authorization, failure, completion, correction, and rollback.

11.2 Viewpoint Design Focus

Viewpoint	Architecture focus	Authority boundary
System context	Local user, local device, CLI, local read-first console, local sources, and optional remote integrations.	Local authority remains explicit across every access surface.
Logical	Kernel coordination over routing, delayed work, context, bounded action use, `PolicyDecision`, `RuntimeJournal`, and recovery.	Runtime authority is not delegated to interfaces, models, actions, or `InferenceOutput`.
Runtime/process	Governed lifecycle from local request to inspection and recovery.	`WorkRecord` transitions remain recorded, inspectable, and recoverable.
Data/knowledge	Governed context and explicit knowledge/learning types.	Source material, `ConversationRecord`, `KnowledgeRecord`, `KnowledgeCandidate`, and `LearningCandidate` remain distinguishable.

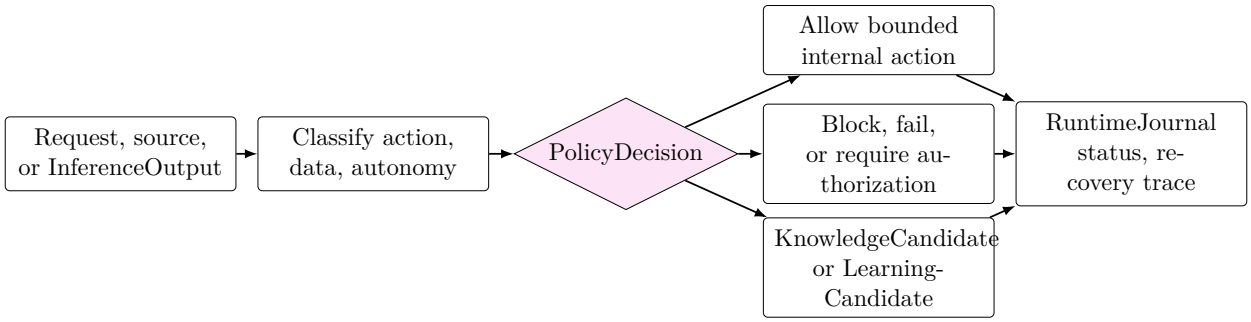
Viewpoint	Architecture focus	Authority boundary
Security/trust	Source-as-data, model-as-function, policy before action, and bounded internal action authority.	Action execution follows declared policy outcomes.
Observability/recovery	Local status, `RuntimeJournal`, verification, policy, knowledge/learning transition, and recovery evidence.	Read-first inspection must not become a privileged mutation path.
Extension	Inference backends, actions, workflows, and learning systems inherit the defined boundaries.	Extension preserves kernel authority and product traceability.

Runtime Governance

12.1 Runtime Authority Rule

Runtime authority belongs to the MADRE kernel and its policy boundaries. User turns, source text, `InferenceOutput`, action results, local inspection, `KnowledgeCandidate`, and `LearningCandidate` can inform decisions, but they cannot authorize their own authority, promote themselves, mutate protected state, or widen permissions. Bounded internal action use requires `PolicyDecision`, local access is read-first where it inspects state, and transitions are recorded through `RuntimeJournal`.

12.2 Policy And Action Gate



12.3 Policy Matrix

Action class	Data class	Autonomy level	Authorization	Trace/Recovery
Read local metadata	Public or low-sensitivity local metadata	Observation	None by default	Log access when used for reasoning.
Draft text	Any allowed context bundle	Draft creation	None unless sensitive scope is crossed	Retain as conversation or generated material, not authority or candidate by itself.

Action class	Data class	Autonomy level	Authorization	Trace/Recovery
Update known material	Scoped module material	Bounded mutation	Required when provenance, truth metadata, scope, or sensitivity changes	Versioned `KnowledgeRecord` with correction and rollback; `KnowledgeCandidate` if boundary change is pending.
Invoke internal action	Small task-scoped non-sensitive input	Bounded internal action	Policy allow required	Requested action, input class, result, policy outcome, and recovery expectation recorded.
Contact remote service	Minimized and classified transfer set	External action	Explicit policy authorization	Transfer record, provider, purpose, and retention assumption recorded.
Modify credentials, identity, legal, financial, destructive, or executable state	Sensitive or critical data	Critical action	Strong human authorization	Block unless recovery or explicit irreversible authorization exists.

Knowledge-boundary updates, routing mutation, workflow mutation, and learning lifecycle behavior follow the same authority rule: a pending knowledge-boundary change is a `KnowledgeCandidate`; an evaluated behavior improvement is a `LearningCandidate`; neither becomes active merely by being produced.

12.4 Learning Lifecycle

Learning follows a controlled lifecycle: observation, evaluation, `LearningCandidate` creation, evidence collection, evaluation, promotion, active monitoring, and rollback. Knowledge handling remains distinct: a stored `KnowledgeRecord` is not learning, and a pending change to its owning knowledge boundary is a `KnowledgeCandidate`. Routing preferences, workflow improvements, and inference-selection improvements remain inactive unless the appropriate learning promotion rule validates them for a defined scope.

Threat Model

13.1 Threat Taxonomy

Threat	MADRE exposure	Primary control	Verification surface
Source as instruction	Untrusted text attempts to alter policy, action execution, or knowledge handling by being interpreted as an instruction.	Source classification and instruction/data separation.	Adversarial context scenario.
Sensitive disclosure	Private context leaks to a model, action, or remote service.	Data classification, minimization, and remote-transfer authorization.	Disclosure review.
Excessive agency	`InferenceOutput` causes over-broad action.	Autonomy levels and action-class policy.	Policy matrix review.
Unsafe action execution	Action invocation produces side effects outside task scope.	Policy-gated internal action boundary; remote, host-command, filesystem, and mutation actions require stronger authority.	Allowed-action and blocked-action scenarios.
Incorrect truth authority for known material	A `KnowledgeRecord` is used as factual authority despite unsuitable `truthLevel` or `truthAuthority`.	Required knowledge metadata, intended-use checks, correction, and reclassification path.	Non-authoritative knowledge-use and truth-correction scenario.
Inspection misuse	Local console is treated as authority or mutation path.	Read-first inspection boundary and shared policy model.	Local inspection negative scenario.

Threat	**MADRE exposure**	**Primary control**	**Verification surface**
Supply-chain or provider dependency	Model, dependency, or service changes behavior or availability.	Replaceable `InferenceBackend` and dependency policy.	Inference substitution review.
Remote transfer risk	Local data is sent outside the user's domain without classification, minimization, authorization, or recorded trace.	Local-first default and remote-transfer policy boundary.	Remote-contact blocked scenario.
Unbounded consumption	Delayed work consumes excessive compute, storage, or cost.	Resource budgets, queue limits, and status inspection.	Resource benchmark.
Trace/recovery gaps	A result cannot be explained, corrected, invalidated, superseded, rolled back, or detached.	`RuntimeJournal`, append-only `RuntimeEvent`, and recovery semantics.	Recovery trace scenario.
Inference output as false authority	Fluent `InferenceOutput` is treated as truth, policy, knowledge, behavior, routing, authority, or a candidate by itself.	Explicit generated-material handling and `PolicyDecision` boundary.	Output non-authority/non-candidate scenario.

Quality Model

14.1 Quality Attribute Scenarios

Attribute	Scenario	Measure or evidence
Responsiveness	The foreground remains conversational while deeper work is queued.	Foreground latency and clear work status.
Recoverability	A failed or unsafe delayed task can be inspected and corrected.	`RuntimeJournal` trace, status transition, and rollback/correction path.
Auditability	A result can be explained through context, `InferenceRecord`, action, `PolicyDecision`, and verification records.	Complete journaled outcome evidence chain.
Maintainability	An `InferenceBackend` or action contract can be replaced without changing kernel authority.	Contract and substitution review.
Portability	A useful subset runs on ordinary local hardware.	Local profile and resource benchmark.
Security	Untrusted context cannot silently become instruction or policy.	Prompt-injection and data-boundary scenarios.
Resource discipline	Delayed work is bounded by cost, time, storage, and compute policies.	Queue and resource-use metrics.
Privacy	Sensitive or unrelated local material is excluded from context and remote transfer unless scoped and policy-allowed.	Context exclusion and remote-transfer scenarios.
Model substitutability	Boundary behavior is preserved across replaceable `InferenceProfile` and `InferenceBackend` choices.	Inference substitution scenario.
Context isolation	One task's context does not silently bleed into another topic, user, inference execution, action invocation, or candidate.	Context scope and revocation review.

Attribute	Scenario	Measure or evidence
Explainability and inspectability	Users and developers can see status, `PolicyDecision`, context evidence, `InferenceRecord`, action evidence, candidates, and recovery state.	Local read-first inspection scenario.
Testability	Architecture claims can be checked through behavior scenarios and negative paths.	Requirements and decision trace review.

UX, Observability, And Recovery

15.1 Interaction Contract

State	User-visible meaning
Immediate answer	The foreground has enough safe local context to answer.
Clarification needed	Intent, scope, risk, or context is insufficient.
Queued reasoning	Work has been recorded for delayed execution.
Running task	Delayed reasoning or action work is active.
Blocked by policy	A policy boundary prevents execution or promotion.
Authorization required	The action crosses a configured authority boundary.
Failed task	Work ended without acceptable result or recovery.
Completed task	Result is available with `RuntimeJournal` trace evidence.
Corrected or rolled back	A previous result or knowledge-boundary change has been revised or reverted.
Invalidated or superseded	Prior evidence, source, or result is no longer active because it was corrected, revoked, reclassified, or replaced.
Deleted or detached	Active generated material is removed from use where allowed while accountable `RuntimeEvent` history remains distinguishable.
Learning candidate	An evaluated `LearningCandidate` proposal exists but is not active.

15.2 Observability Surfaces

The local inspection surface must expose sessions, `WorkRecord`, `RuntimeEvent`, context bundles, `InferenceRecord` and `InferenceOutput`, internal action evidence, `PolicyDecision`, failures, metrics, checkpoints, rollbacks, invalidations, supersessions, deleted/detached active material, `KnowledgeCandidate`, `LearningCandidate`, and recovery actions. The inspection surface is not a privileged bypass; it is a read-first view over the same policy-governed runtime.

Evaluation And Benchmark Specification

MADRE defines an architectural evaluation surface for testing whether the runtime preserves local authority, governed context, bounded action use, evaluated learning promotion, journaled traceability, and recoverable delayed reasoning.

16.1 Benchmark Families

Benchmark family	Purpose
Functional correctness	Confirm expected runtime behavior for foreground-lane continuity and delayed work.
Delayed reasoning quality	Evaluate evidence handling, decomposition, verification, and result delivery.
Foreground triage	Confirm unsupported evidence-heavy answers are clarified or delayed rather than fabricated.
Context isolation	Confirm unrelated or sensitive knowledge does not enter the wrong bundle.
Inference substitution	Confirm replaceable `InferenceProfile` and `InferenceBackend` choices preserve the same boundary behavior and expose failures.
Policy enforcement	Confirm action, data, autonomy, authorization, and recovery rules are enforced.
Bounded internal action	Confirm the bounded internal action is policy-gated, recorded, and harmless.
Blocked unsafe actions	Confirm remote, host-command, filesystem, mutation, and critical-action attempts do not execute under bounded internal-action authority.
Inference output non-authority	Confirm `InferenceOutput` remains generated material, neither authority nor a candidate by itself.
Known-material truth control	Confirm a `KnowledgeRecord` with non-authoritative truth metadata is not used as factual truth.

Benchmark family	Purpose
Learning promotion control	Confirm a `LearningCandidate` remains inactive until evaluated promotion and that saving `KnowledgeRecord` is not learning.
Runtime journal traceability	Confirm `WorkRecord`, `RuntimeEvent`, `PolicyDecision`, and inference references expose lifecycle and recovery evidence locally.
Read-first observability	Confirm CLI/status and local console inspection expose lifecycle, policy, context, inference, action, knowledge/learning transition, and recovery evidence.
Recovery	Confirm failed or unsafe outputs can be inspected, corrected, deleted/detached, invalidated, superseded, or rolled back.
Resource use	Measure latency, storage, queue time, model cost, and local compute use.
Reproducibility	Preserve enough configuration, context, and event history to review outcomes.
Hallucination resistance	Confirm `InferenceOutput` does not become truth, knowledge, action authority, routing, behavior, or candidate status by itself.

Governance, Deployment, And Limits

17.1 Secure Engineering Principles

MADRE requires secure defaults for local credentials, dependency selection, governed external boundaries, secrets handling, reviewable release records, and vulnerability response. Security evidence must cover both conventional software risks and model-mediated risks such as prompt injection, poisoned retrieval, unsafe output handling, excessive agency, and remote disclosure.

17.2 Open-Source Governance

MADRE is released under `GPL-3.0`. The open-source governance model must preserve license clarity, dependency review, contribution rules, security disclosure, release integrity, and community responsibility. Openness is part of the project value only when users and developers can inspect, adapt, and verify the runtime without surrendering private data to a platform owner.

17.3 Deployment And Operational Assumptions

The baseline deployment target is an ordinary local device with local storage, local configuration, optional local model execution, and optional remote integrations governed by policy. The operational profile includes storage paths, model setup assumptions, backup and restore, migration, diagnostics, log retention, offline behavior, and upgrade or rollback expectations.

Deployment assumptions remain conceptual at this level of specification. The product architecture defines authority, locality, inspection, recovery, and model-runtime and action boundaries without reducing deployment to a single operational shape.

17.4 Risks, Limitations, And Excluded Functions

MADRE limits active executable behavior by authority class. Workflow authoring, self-learning, speech interaction, graphical product surfaces, distributed inference, organization-scale deployment, and model fine-tuning remain governed by the same local-first, policy-governed, model-agnostic, auditable, and recoverable runtime boundary as the rest of the product.

Requirements Catalogue

ID	Requirement summary	Decision trace
FR-001	Preserve foreground-lane conversation while delaying work that requires evidence, verification, internal actions, or stronger context.	D-001, D-004.
FR-002	Record delayed reasoning as `WorkRecord` with `RuntimeEvent` lifecycle and recovery path through `RuntimeJournal`.	D-001, D-003, D-008.
FR-003	Construct context bundles from sourced, classified, minimized, scoped, provenanced, and revocable information.	D-001, D-005, D-007.
FR-004	Execute model-backed work through `ModelAction`, allowed `ModelBinding`, and replaceable `InferenceBackend`, producing `InferenceRecord` and non-authoritative `InferenceOutput`.	D-001, D-006.
FR-005	Expose executable behavior only as declared, bounded, traceable internal actions or explicitly authorized external actions.	D-007, D-009.
FR-006	Keep pending `KnowledgeCandidate` change separate from evaluated `LearningCandidate` improvement and require promotion for each.	D-011.
FR-007	Expose local request entry and read-first inspection.	D-010.
FR-008	Represent recovery outcomes explicitly.	D-003, D-008.
SEC-001	Treat prompt/source material as data rather than instruction; treat `InferenceOutput` as generated material rather than authority or candidate by itself.	D-004, D-005, D-007.
OPS-001	Keep local access surfaces governed by runtime authority.	D-010.

Traceability Matrix

Requirement	Concern	Architecture element	Evidence family
FR-001	Responsiveness and user control	Complex Agent foreground lane and delayed reasoning split	Interaction and latency scenarios.
FR-002	Recoverability and traceability	`WorkRecord`, `RuntimeEvent`, and `RuntimeJournal`	Recovery and status scenarios.
FR-003	Privacy and correctness	Governed context bundle	Context isolation scenarios.
FR-004	Provider independence	`ModelAction`, `InferenceBackend`, `InferenceRecord`, and `InferenceOutput`	Inference substitution and output-boundary scenarios.
FR-005	Action safety	Internal Action Registry, action contract, and external action policy	Allowed-action and blocked-action scenarios.
FR-006	Controlled evolution	Separate knowledge-change and learning-promotion candidates	Knowledge-boundary and learning-promotion review.
FR-007	Local operability	CLI plus read-first local console	Local access and inspection scenario.
FR-008	Recoverability	`RuntimeEvent` history and recovery semantics	Recovery trace.

Threat Catalogue

Threat family	**Primary boundary**	**Initial response**
Source as instruction	Context bundle construction	Separate source material from instruction and policy.
Sensitive disclosure	Data classification and remote transfer	Minimize, redact, authorize, and record disclosures.
Unsafe action execution	Action and workflow contracts	Require action-class policy, bounded internal-action scope, and recovery expectation.
Known material used under incorrect truth metadata	`KnowledgeRecord` use	Enforce provenance, `truthLevel`, `truthAuthority`, intended use, correction, and reclassification.
Inspection bypass	Local console and status surfaces	Read-first inspection with no privileged mutation path.
Inference-output false authority	`InferenceOutput` boundary	Treat `InferenceOutput` as generated material, not authority or candidate by itself.
Provider dependency	Local Model Runtime Boundary	`InferenceBackend` substitution and local-first default.
Unbounded consumption	Scheduler and resource controls	Queue, compute, storage, and cost limits.

Benchmark Catalogue

Benchmark	Observed property	Evidence
Foreground response	Conversational continuity under delayed work	Latency and interaction-state record.
Delayed work durability	`WorkRecord` survives interruption and exposes status	`RuntimeEvent` lifecycle in `RuntimeJournal`.
Context isolation	Only scoped material with suitable metadata enters a task context	Context bundle and `KnowledgeRecord` inspection.
Policy enforcement	Disallowed action remains blocked or requires authorization	Policy decision record.
Bounded internal action	Bounded internal action remains local, policy-gated, and traceable	Action trace.
Inference output boundary	`InferenceOutput` remains generated material, not authority or candidate by itself	`InferenceRecord`, output-handling, and negative-path trace.
Knowledge truth authority	Non-authoritative `KnowledgeRecord` is not used as factual truth	Truth-metadata use and correction trace.
Learning promotion control	`LearningCandidate` remains inactive and storing `KnowledgeRecord` is not learning	Promotion and negative-path trace.
Runtime journal traceability	Local trace correlates work, events, policies, inference, and recovery	`RuntimeJournal` inspection trace.
Read-first inspection	Local console exposes status, trace, policy, context, inference, action, knowledge/learning transition, and recovery evidence	Inspection trace.
Recovery	Failed or unsafe result can be corrected or rolled back	Recovery trace.
Inference substitution	`InferenceBackend` replacement preserves the boundary contract	Inference boundary comparison.

Runtime Domain Model

This appendix is authoritative for **MADRE** architecture vocabulary and semantic boundaries. It is explicitly not a Java package layout, persistence schema, method-signature catalogue, API wire shape, or inheritance strategy. An implementation may choose its structure only if it preserves these concepts, ownership boundaries, and authority rules.

E.1 Coordination And Module Ownership

Domain term	Architecture meaning
`MadreKernel`	Deterministic coordinator that registers `ReasoningModule` objects, assigns the Default interaction module, routes requests, schedules delayed work, coordinates resources, applies policy boundaries, and records runtime activity through `RuntimeJournal`. It does not reason, prompt directly, learn, own knowledge truth, or execute inference directly.
`ReasoningModule`	Governed reasoning unit that owns bounded scope, module records, workflows, routines, actions, concrete `MADREAgent` objects, knowledge/learning boundaries, and permitted model/inference scope.
Default interaction module	Registration role for the compatible `ReasoningModule` that receives ordinary interaction first. It is not a distinct class or required module type.
`ModuleMainAgent`	Module role held by one `MADREAgent` for ordinary module coordination. A `ComplexAgent` may fulfill this role, but the two terms are not synonymous.
`MADREAgent`	Concrete runtime actor owned by one `ReasoningModule`; it operates within module authority and permitted `ModelBinding`. Reusing an agent definition elsewhere does not carry knowledge or learning across the module boundary.
`ComplexAgent`	Optional agent capability profile for foreground, reasoning, and reflection work. It grants no additional policy, knowledge, or action authority.

E.2 Work, Actions, And Inference

Domain term	Architecture meaning
`ReasoningPlan`	Runtime plan carrying objective, target module, scope, constraints, context references, execution references, correction, rollback, and evaluation material; it may select a `Workflow`.
`Workflow` / `AgentRoutine`	Reusable module-level or agent-level execution structure used by a plan under module and kernel authority; neither is an independent authority or second plan.
`AgentAction` / `ModelAction`	Executable capability and its inference-backed specialization. A `ModelAction` prepares governed input and executes only through the agent's allowed `ModelBinding`.
`ModelBinding`	Allowed inference configuration for a module-owned agent or its model-backed action, constraining available `InferenceProfile`, model files, and `InferenceBackend`.
`InferenceProfile` / `InferenceBackend`	Inference execution configuration and replaceable execution boundary respectively. Neither is a model authority layer.
`InferenceRecord`	Passive record of one inference execution, including profile, backend, configuration, timing, status, and output reference.
`InferenceOutput`	Generated material produced by an `InferenceRecord`. It is not truth, policy, memory, knowledge, behavior, routing, authority, or a candidate by itself.

E.3 Trace, Context, Knowledge, And Learning

Domain term	Architecture meaning
`WorkRecord`	Durable delayed-work lifecycle record linked to its plan, module, request, runtime events, and recovery references. It is not plan state, knowledge state, candidate state, or learning state.
`RuntimeEvent` / `RuntimeJournal`	Append-only history of runtime activity and the local trace mechanism that records work state, runtime history, and related payload references. They do not create a separate authority subsystem.
`PolicyDecision`	Explicit allow, block, or authorization-required outcome with reason, affected action/data class, authority boundary, and related `RuntimeEvent`. Generated material cannot create or widen it.
`ContextBundle`	Governed, selected, minimized, classified, and provenanced context for a request, action, or inference execution. Included source material does not become instruction.
`KnowledgeRecord`	Material known by a module with provenance, scope, sensitivity, intended use, `truthLevel`, `truthAuthority`, correction, and lifecycle metadata. It is not factual truth by default.
`KnowledgeCandidate`	Pending change to a module knowledge boundary, such as inclusion, transfer, reclassification, or truth-authority change. Accepted known material is a `KnowledgeRecord`.
`LearningCandidate`	Evaluated proposed module improvement that may affect behavior, evaluation, routing, routines, workflows, context selection, trust calibration, or inference selection. Saving a `KnowledgeRecord` is not learning.
`ConversationRecord`	Retained interaction material for continuity, traceability, or later evaluation. It is not knowledge by default and only becomes known material through explicit handling.

Glossary

Term	Definition
Internal action	A typed runtime-defined executable function available to modules through the Internal Action Registry with declared inputs, outputs, side-effect class, policy requirements, trace obligations, and recovery expectations.
Default interaction module	A `ReasoningModule` role assigned by kernel registration for ordinary interaction first routing; it is not a module type.
ModuleMainAgent	The role held by one module-owned `MADREAgent` for ordinary module coordination; a `ComplexAgent` may fulfill it.
ComplexAgent	An optional `MADREAgent` capability profile that may coordinate foreground, reasoning, and reflection lanes without gaining additional authority.
ModelAction	An `AgentAction` that uses governed context and an allowed `ModelBinding` to execute inference through `InferenceBackend`.
ModelBinding	Allowed model/inference configuration assigned to a module-owned agent or model-backed action.
InferenceProfile	Configuration recipe for permitted inference execution; it is neither the model nor authority.
InferenceBackend	Replaceable execution boundary for local or explicitly authorized inference software.
InferenceRecord	The passive record of one inference execution and its configuration, backend, status, timing, and output reference.

Term	Definition
InferenceOutput	Generated material from an `InferenceRecord`; it is not truth, policy, memory, knowledge, behavior, routing, authority, or candidate by itself.
Local Model Runtime Boundary	Product-level phrasing for controlled local or authorized inference execution through `ModelBinding`, `InferenceProfile`, `InferenceBackend`, `InferenceRecord`, and `InferenceOutput`.
WorkRecord	Durable delayed-work lifecycle record linked to plan, module, request, runtime events, and recovery references.
RuntimeEvent	Append-only record of runtime activity, including policy, action, inference, correction, and recovery transitions.
RuntimeJournal	Local runtime trace mechanism that records `WorkRecord`, appends `RuntimeEvent`, and stores or references related payloads.
PolicyDecision	Explicit allow, block, or authorization-required outcome for an affected authority boundary.
KnowledgeRecord	Module-owned known material with provenance, scope, sensitivity, intended use, truth metadata, correction, and lifecycle semantics; it is not factual truth by default.
KnowledgeCandidate	A pending change to a module knowledge boundary, not ordinary stored material.
LearningCandidate	An evaluated proposed module improvement; saving a `KnowledgeRecord` is not learning.
ConversationRecord	Retained interaction material for continuity, traceability, or evaluation; it is not knowledge by default.

Term	Definition
Read-first console	A local inspection surface that exposes status and `RuntimeJournal` trace without becoming a privileged mutation path.
Recovery path	The defined correction, rollback, deletion, compensation, or blocking behavior for an outcome or state transition.
External tool	A non-canonical industry term for optional integrations outside the default **MADRE** internal-action path. External tools and services are higher-risk external actions requiring explicit policy, trace, and recovery boundaries.

Product Boundary Statement

MADRE is a local-first, model-agnostic runtime kernel for governed agentic systems. Product behavior is defined by the architecture, requirements, safety boundaries, traceability, and validation criteria in this specification.